

Guía Esencial de Git y GitHub

Manual práctico para estudiantes de desarrollo de software

1. Gestión de Ramas (Branches)

Las ramas permiten trabajar en nuevas funcionalidades o experimentos sin afectar el código de producción.

```
main:    o---o---o---o
          \   /
feature:  o---o
```

Crear y moverte a una nueva rama

El comando más rápido para crear una rama y cambiarte a ella en un solo paso:

```
git checkout -b feature-database-model
# Crea la rama 'feature-database-model' y te posiciona en ella
```

Ver todas las ramas

```
git branch
# La rama actual tendrá un asterisco (*)
```

Subir la nueva rama a GitHub

Si acabas de crear la rama localmente, debes publicarla en el repositorio remoto:

```
git push -u origin feature-database-model
```

2. Configurar Permisos (Solo Lectura/Descarga)

Aclaración técnica: Git por sí mismo no tiene un comando de terminal para hacer una rama "solo lectura". El control de permisos se gestiona directamente en la plataforma web de **GitHub** mediante *Branch Protection Rules*.

Configuración en GitHub para restringir escritura:

Para que los estudiantes o colaboradores solo puedan ver y descargar (clonar/pull) el código de una rama sin poder modificarla, sigue estos pasos:

1. Ve a tu repositorio en **GitHub.com**.
2. Haz clic en **Settings** (Configuración) en la barra superior.
3. En el menú lateral izquierdo, selecciona **Branches**.
4. Haz clic en el botón **Add branch protection rule**.
5. En *Branch name pattern*, escribe el nombre de la rama que deseas proteger.
6. **Activa las siguientes opciones:**
 - **Lock branch**: Esto hace que la rama sea de solo lectura. Nadie podrá hacer *push* directo a esta rama.
 - **Require a pull request before merging**: Exige que cualquier cambio pase por una revisión antes de integrarse.
7. Guarda los cambios haciendo clic en **Create**.

Resultado: Los usuarios podrán hacer `git clone` o `git pull` para descargar el código sin problemas, pero si intentan hacer `git push`, GitHub rechazará la subida.

3. Hoja de Trucos: Comandos Comunes

Comando	¿Para qué sirve?
<code>git init</code>	Inicializa un nuevo repositorio de Git en la carpeta actual.
<code>git clone <url></code>	Descarga un repositorio de GitHub a tu computadora local.
<code>git status</code>	Muestra el estado actual de tus archivos (modificados, agregados, sin seguimiento).
<code>git add .</code>	Agrega todos los archivos modificados al área de preparación (staging area).

Comando	¿Para qué sirve?
<code>git commit -m "msj"</code>	Guarda los cambios preparados con un mensaje descriptivo de la versión.
<code>git push origin <rama></code>	Sube tus <i>commits</i> locales a la rama especificada en GitHub.
<code>git pull origin <rama></code>	Descarga e integra los últimos cambios de GitHub a tu rama local.
<code>git log</code>	Muestra el historial de <i>commits</i> realizados en la rama actual.
<code>git merge <rama></code>	Fusiona la <rama> indicada dentro de la rama en la que te encuentras posicionado.